

Faster Lance Index Builds on Kunpeng

ARM SVE/SVE2 kernels · tree assignment · PiPNN partition graphs

Project proposal v6 · 8 November 2026

Vector Index Optimization

HUAWEI

Three build wins measured; Kunpeng kernels next

[x86] measured by us, equal recall · [Kunpeng TBD] filled after native runs · the wins are dense-distance loops WP1 ports to SVE

WP2 · TREE

2.5×

IVF_RQ · BIGANN10M · K 16,384

· 37.7 s → 15.1 s
· assignment CPU 12.8× lower
· recall@10 within 0.005 of exact

X86 MEASURED

WP3 · PIPNN GRAPHS

4.9×

vs HNSW · BIGANN10M · K 5

· 537.7 s → 109.9 s, 6.9× less CPU
· index 15% smaller
· recall higher at 9 of 10 settings

X86 MEASURED

EXISTING PATCH

5.7×

IVF_RQ RQ9 · GIST1M · K 1,000

· 221.0 s → 38.5 s
· 4×4 tile alone: 2.07× at RQ1
· R@10 change ≤ 0.006

X86 MEASURED

WP1 · KUNPENG (LEAD)

TBD

Same builds, Kunpeng SVE/SVE2

· 6 SVE/SVE2 kernels ready (QEMU)
· tiled L2 + argmin, leaf Gram next
· x86 win needs these tiles on ARM

PLACEHOLDER

Why kernels lead: all three wins replace scattered, per-vector distance work with dense tiles. On x86 those tiles exist (AVX-512). On Kunpeng, Lance has pairwise kernels only, so the same algorithms would run on a slower path.

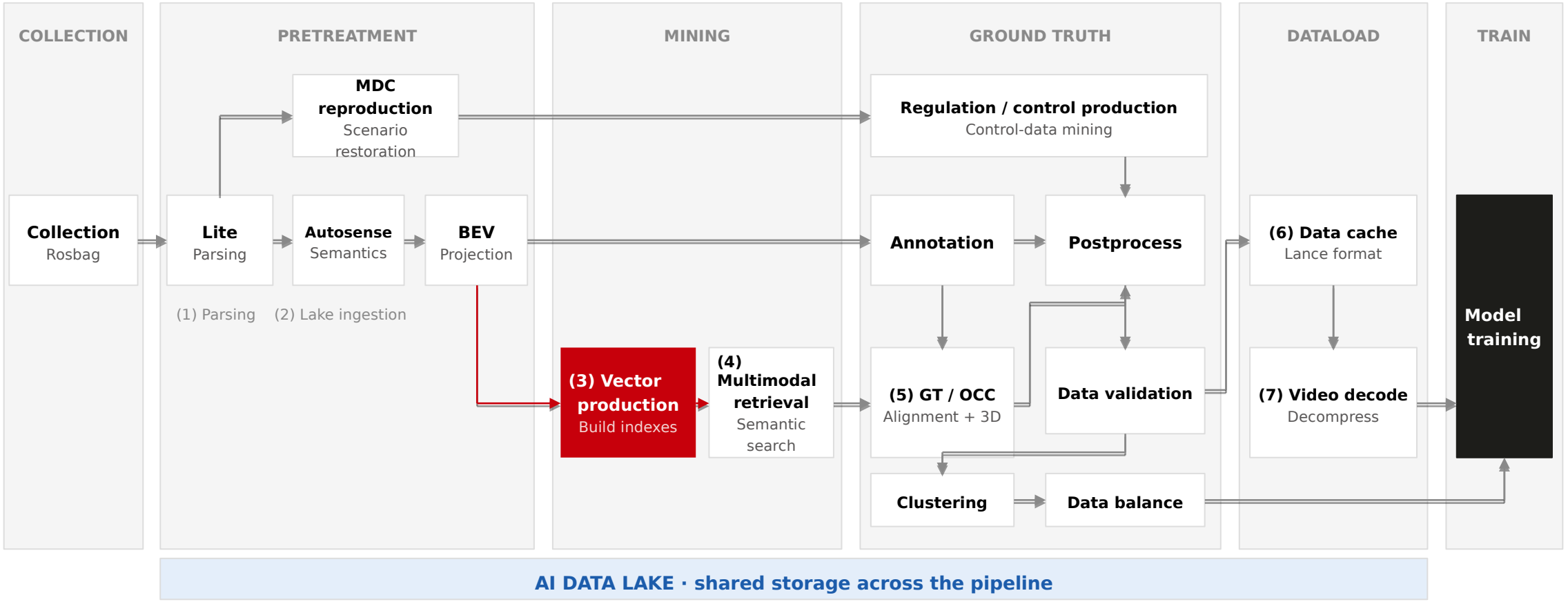
Ask: Kunpeng server time (SVE/SVE2), owners for WP1–WP3, and an agreed recall tolerance (proposed ±0.005 recall@10).

Measured on x86, ready for Kunpeng. The remaining unknown is ARM kernel speed, which is what WP1 resolves.

– Huawei Confidential –

Target (3): faster index construction

[Company context] Redrawn from ADS-PLAN slide 1; target is Vector Production → Building Vector Indexes.



Index construction is the intervention point. Faster builds can reduce delay before multimodal retrieval.

– Huawei Confidential –

Build time sits in three dense-distance loops

[x86] stage spans from our runs · bars are each build's own total (100%) · labels in seconds

WHERE THE SECONDS GO

IVF_RQ · BIGANN10M · K = 16,384 — fixed centroids (training 17.7 s excluded)



→ Assignment: 761 CPU-s = 16,384 distances × 10M rows

IVF_RQ RQ9 · GIST1M · K = 1,000 — 4-core Xeon, upstream Lance



→ Encoding: per-vector scale search for multi-bit codes

IVF_HNSW_FLAT · BIGANN10M · K = 5 — per-partition graphs of 1.1–3.2M rows



→ Graph build: one beam search per inserted vector

LOOP → WHAT FIXES IT

1 · Nearest-centroid search

k-means + full-data assignment

Fix: WP2 tree routing · WP1 tiled L2 + argmin

2 · Multi-bit RQ encoding

scale search per vector

Fix: constant scale (existing patch)

3 · Partition graph build

HNSW insertion search

Fix: WP3 PiPNN · WP1 leaf Gram tiles

All three are dense L2 / dot-product math over many vectors. On x86, Lance and our patches run them on AVX-512 tiles. On ARM, Lance has pairwise NEON kernels plus our six SVE ports, and no tiles.

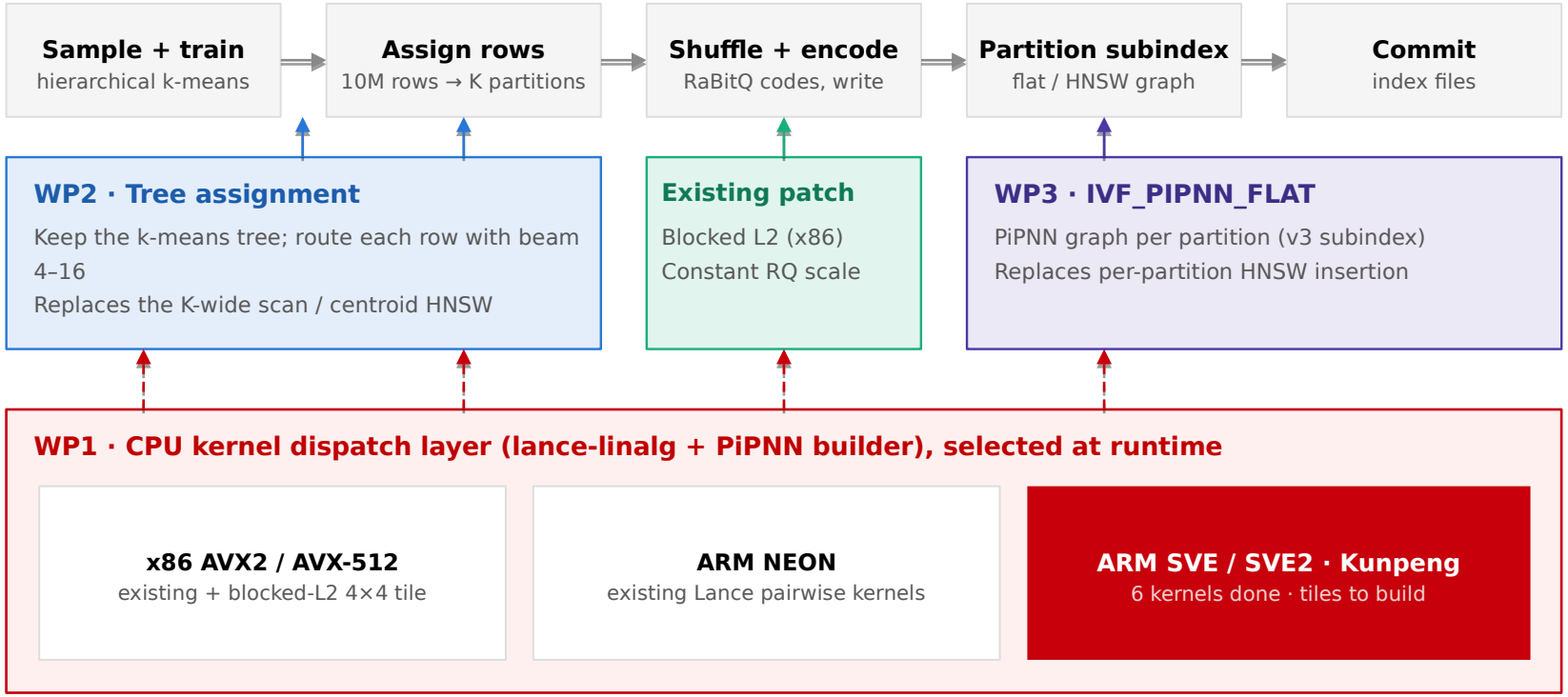
The bottleneck is the same arithmetic everywhere. One kernel layer speeds up all three loops.

– Huawei Confidential –

One project: three work packages, one kernel layer

Lance IVF build pipeline with each work package at its integration point · solid = implemented on x86 · dashed red = to build for Kunpeng

LANCE INDEX BUILD (CREATE_INDEX)



STATUS · 8 NOV 2026

WP1 kernels

· 6 SVE/SVE2 kernels: clang-18 -Werror, QEMU 128–2048-bit VL
· Tiled L2 + argmin: x86 done, SVE in progress
· Native Kunpeng run: next

WP2 tree

· Retained tree + beam routing in Lance branch
· x86 suites A/B measured

WP3 PiPNN

· IVF_PIPNN_FLAT builds, persists, searches
· x86 suite C measured (build + recall)

WP2/WP3 change what is computed; WP1 makes it fast on Kunpeng. Each ships separately.

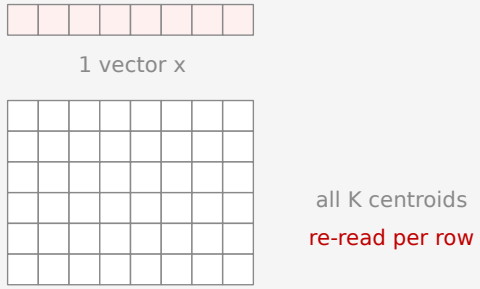
– Huawei Confidential –

Tiling the distance loop doubled x86 build speed

Proposal 1 · WP1 ARM kernels · [x86] GIST1M, K = 1,000, RQ1, 4-core Xeon, 2 repeats · our blocked-L2 patch vs upstream Lance

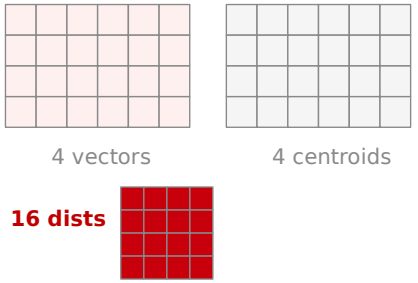
SAME MATH, DIFFERENT MEMORY TRAFFIC

UPSTREAM · PAIRWISE



1,000 × 960 f32 = 3.8 MB table streamed from memory for every vector

PATCH · 4×4 TILE

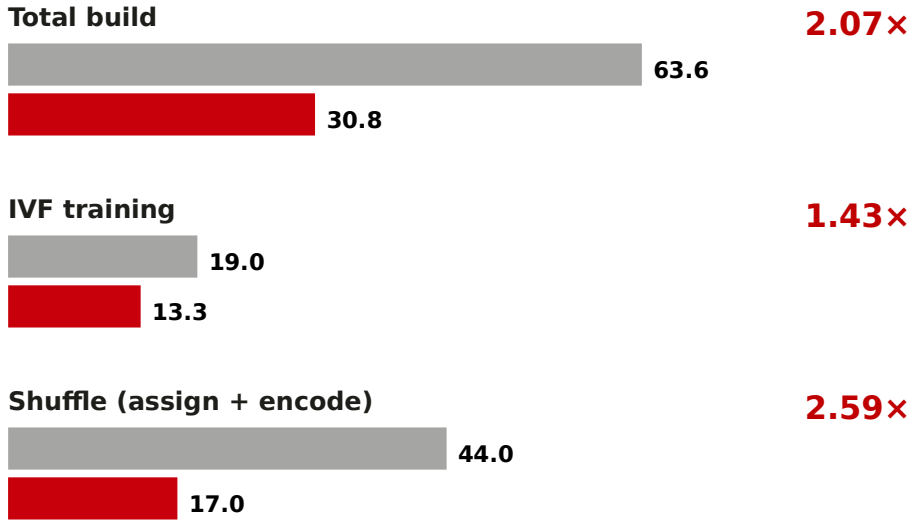


each centroid load reused 4×, centroid chunks sized to L2

On Kunpeng this win does not exist yet. The patch is AVX-512 / AVX2+FMA only; on ARM the same code falls back to the per-vector path. WP1 writes the SVE tile.

rows being assigned · centroid block in cache · output distance tile

MEASURED · SECONDS (UPSTREAM → PATCH)



Recall@10 unchanged within noise (0.610 → 0.614 at nprobes 20). Lance K-means and RaBitQ tests pass (86/86).

The speedup came from data reuse, not wider registers. That transfers to SVE: the tile shape is ISA-independent.

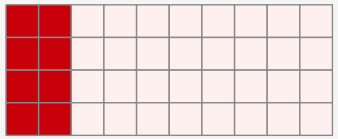
– Huawei Confidential –

HUAWEI

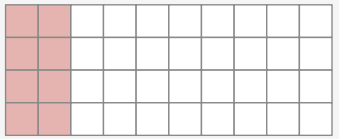
One tiled L2 + argmin kernel serves every new loop

Proposal 1 · WP1 · method figure drawn by us · SVE is vector-length agnostic: one binary runs at 128–2048-bit VL · sizes illustrative

① Register tile over D, in VL-sized steps



X block: 4 rows × D

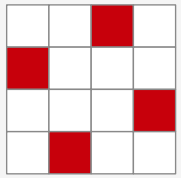


C block: 4 centroids × D

$acc_{ij} \mathrel{+}= \Sigma\ (x_i \cdot c_j)$ over one VL chunk (red columns)

· 16 accumulators stay in SVE Z registers; each loaded chunk is used 4×
· svwhilelt predicate covers the tail: no scalar remainder, no over-read
· $d^2 = \|x\|^2 + \|c\|^2 - 2\ x \cdot c$, norms precomputed once per block

② Fused epilogue



4×4 distances

· + balance bias (k-means)
· running argmin or small top-k
· first strictly smaller wins; NaN rows stay unassigned
· chosen distance recomputed exactly for RaBitQ residuals

③ Consumers

k-means training
rows × K centroids

Tree beam step (WP2)
rows × 16·beam children

PiPNN leader routing (WP3)
rows × leaders

PiPNN leaf k-NN (WP3)
leaf Gram L×L, L = 256–1024

Contracts carried over from x86: same winner rule as upstream, exact selected distance, scalar fallback when SVE is absent, objects built with -msve-vector-bits=scalable and no global -march, vector length queried per call. Tests: ties, NaN, tails, irregular strides, 5 vector lengths, guard pages.

Write the tile once, use it four times. k-means, tree routing and both PiPNN hot loops share it.

– Huawei Confidential –

Six SVE kernels ready; tiled kernels are next

Proposal 1 · WP1 · status of every ARM kernel in scope · Done = QEMU-validated at 128–2048-bit VL, native Kunpeng timing pending

Done = implemented, emulator-validated **P0** = needed by WP2/WP3 hot loops **P1 / P2** = after profiling on Kunpeng

Pri	Kernel (ARM)	Serves	Status	Kunpeng result
Done	f32 dot + squared L2, pair and batch (SVE FMA)	HNSW / flat search, k-means today	QEMU ✓ · dispatcher ✓	**TBD**
Done	u8 dot (SVE UDOT) + u8 squared L2	uint8 vectors (BIGANN)	QEMU ✓ · overflow tests	**TBD**
Done	4-bit LUT accumulate (SVE gathers; SVE2 TBL high-accuracy)	PQ / RQ fast scan	QEMU ✓ · exact order	**TBD**
P0	Tiled f32 L2 + fused argmin / top-k + bias	k-means, tree beam, PiPNN routing	x86 version measured (2.07×); SVE in progress	**TBD**
P0	Leaf Gram / L2 tiles + top-k, bounded scratch	PiPNN leaf k-NN	x86 AVX-512 in PiPNN; SVE planned	**TBD**
P1	Batched candidate L2 + row gather	RobustPrune, graph search	planned	—
P1	Projection sketches, sign compare, u16 hash	PiPNN HashPrune	planned	—
P2	Small-k selection; u8 integer distance tiles	byte-input leaf builds	planned	—

ALREADY VERIFIED

· clang-18 -Wall -Wextra -Werror · QEMU at 5 vector lengths, VL changed mid-process
· guard pages expose tail over-reads · Rust dispatcher selects SVE only when the OS reports it

NOT YET

· native Rust build + full Lance tests on ARM (toolchain certificate issue on our host)
· any Kunpeng timing · SME (deferred until profiles justify it)

The pairwise base is done; the tiles are the project. P0 rows decide whether WP2/WP3 gains survive on Kunpeng.

– Huawei Confidential –

Kunpeng runs reuse frozen configs, five ablations

Proposal 1 · WP1 · [Proposal] evaluation design · placeholders fill in as runs complete · speedups computed within one machine only

ABLATION LADDER (SAME KUNPENG HOST)

1. **Stock Lance**
   reference
2. **+ existing IVF_RQ patch**
   blocked L2 falls back on ARM
3. **+ existing SVE kernels**
   pairwise SVE vs NEON
4. **+ WP2 tree / WP3 PiPNN**
   algorithm gain, old kernels
5. **+ new P0 SVE tiles**
   kernel gain on new loops

RESULTS TABLE · TO BE FILLED

Measurement	x86 EPYC (ours)	Kunpeng 1–3	Kunpeng 5
IVF_RQ build, BIGANN10M K = 16,384, exact	37.7 s	**TBD**	**TBD**
… with tree:4 assignment (WP2)	15.1 s	**TBD**	**TBD**
IVF_HNSW_FLAT build, BIGANN10M K = 5	537.7 s	**TBD**	—
IVF_PIPNN_FLAT build, same (WP3)	109.9 s	**TBD**	**TBD**
Tiled L2 + argmin, D = 128 / 960 (GFLOP/s)	x86 tile exists	**TBD**	**TBD**
Pair kernels: SVE vs NEON (Criterion)	—	**TBD**	—
Recall@10 change vs exact / HNSW	≤ 0.005	**TBD**	**TBD**

Recorded per run: Kunpeng model, SVE vector length, cores / NUMA, affinity, frequency policy, compiler, thread counts.

Gate for each new kernel [Proposal]: faster than the existing NEON/SVE path on the same Kunpeng host in microbenchmark and in full build, with recall@10 within ±0.005. Otherwise the dispatcher keeps the old path. x86 vs Kunpeng seconds are context, never a speedup.

Every Kunpeng number will have a same-machine baseline. The harness and configs are frozen and run unchanged.

– Huawei Confidential –

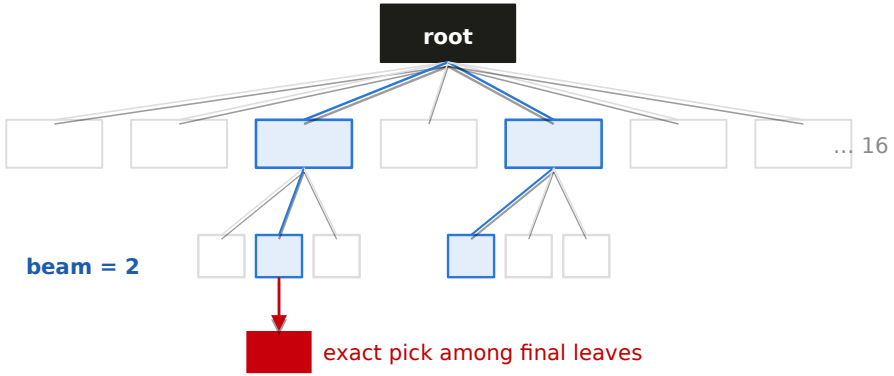
Reuse the trained tree to assign rows

Proposal 2 · WP2 tree assignment · hierarchical k-means, 16 children per node, K = 16,384 → 3–4 levels · tree drawn smaller

TODAY · TREE THROWN AWAY

· k-means builds a 16-way tree, then keeps only the K leaf centroids
· Assignment: scan all K (16,384 distances / row), or a centroid HNSW when K·D ≥ 1M (Lance default)
· HNSW must be built and loses ~4–5% of exact winners at default ef

PROPOSED · ROUTE DOWN THE KEPT TREE



Keep the beam closest nodes per level → exact distance to the chosen centroid

SAMPLE OF 100K ROWS [X86]

Method	Time	Match
exact	0.248 s	1.000
tree:4	**0.023 s**	**0.917**
tree:16	0.055 s	0.997
HNSW (default)	1.28 s*	0.952

* + 0.11 s graph build. Tree prep 0.002 s: training already made it. Match = same centroid as exact.

Distance work per row ≈ beam × 16 per level instead of K. With K = 16,384 and 3–4 levels, beam 4 touches a few hundred centroids instead of 16,384, and every touch is a small dense block (WP1 tile).

Why the occasional wrong leaf is cheap

Queries still probe nprobes partitions chosen exactly; a near-boundary row in a neighbouring partition is still found. Measured below: recall unchanged.

What stays unchanged

Query routing, index format and centroid layout. The tree is build-only state, not persisted. Exact / auto remain selectable.

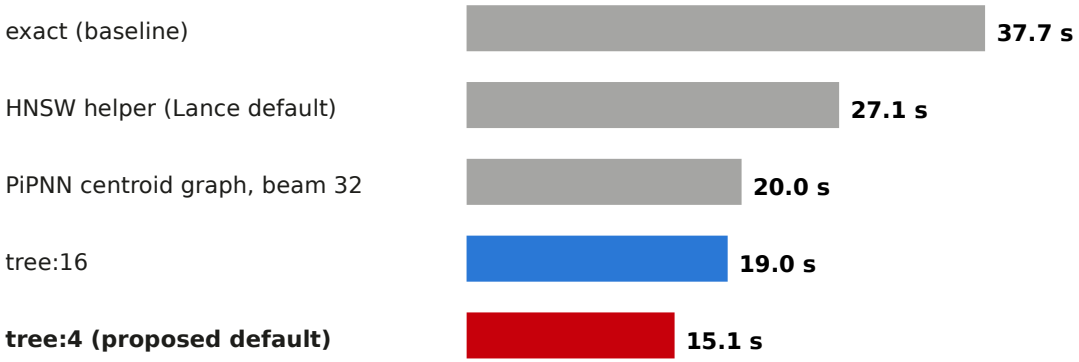
The tree is free and already trained. Using it removes most assignment work without new index state.

– Huawei Confidential –

Tree assignment cuts BIGANN10M build time 2.5×

Proposal 2 · WP2 · [x86] IVF_RQ 5-bit, K = 16,384, fixed centroids (only assignment differs), 1,970 tuning queries, 1 run each

BUILD SECONDS · BIGANN10M (LOWER IS BETTER)



QUALITY AT THE SAME SETTINGS

Method	vs exact	vs default	Match	R@10 np=64
exact	1.00×	0.72×	1.000	0.9066
HNSW default	1.39×	1.00×	0.952	0.9026
PiPNN graph	1.89×	1.35×	0.993	0.9061
tree:16	1.99×	1.43×	0.997	0.9083
tree:4	**2.49×**	**1.79×**	**0.917**	**0.9062**

Assignment CPU: 760.9 → 59.4 thread-s (12.8× less). Query latency unchanged (p50 within ±1 ms).

OTHER DATA · WHERE IT HELPS LESS

GIST1M, K = 16,384

· complete builds (k-means retrained)
· exact 31.4 s · Lance auto 24.3 s
· tree:4 23.0 s (1.37×) · tree:1 19.9 s
· training (~9.5 s) now dominates

GIST1M, K = 1,000

· K·D below the helper threshold
· exact 11.7 s vs tree:4 11.6 s
· no gain: assignment already cheap
· auto keeps exact at small K

Centroid graphs lose

· HNSW and PiPNN over centroids are slower than the tree at equal match
· graph route not pursued for assignment
· (PiPNN stays for partition graphs, WP3)

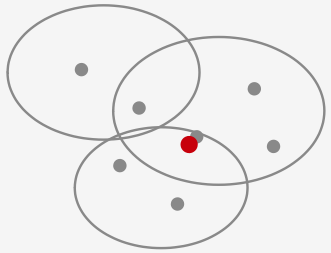
Biggest gain at large K with many rows: the ADS case. At small K, Lance's exact path stays.

– Huawei Confidential –

PiPNN swaps insertion search for leaf matrix math

Proposal 3 · WP3 · drawn by us after Rubel et al., KDD 2026 §3–4 · red tags = where WP1 kernels plug in · sizes illustrative

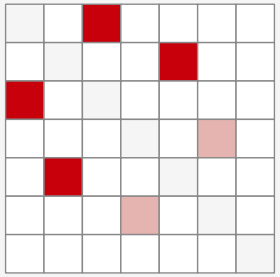
1 · Overlapping leaves



Random leaders; each point joins a few nearest → leaves ≤ 1,024

P0 TILE: ROUTING

2 · Leaf Gram → local k-NN



All pairs inside a leaf as one dense matrix; keep k nearest each way

P0 TILE: GRAM

3 · HashPrune

c1 1.2

c2 2.0

c3 1.5

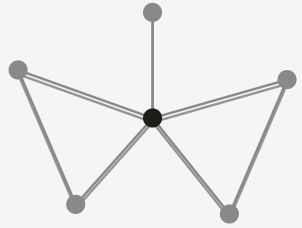
c4 3.1 evicted

c5 0.9

Bounded reservoir per point keeps near candidates in varied directions

P1: SKETCHES

4 · RobustPrune



Final degree-bounded graph; search starts at the medoid

P1: BATCHED L2

HNSW (TODAY) · PER-VECTOR INSERTION

Each new vector runs a beam search over the graph built so far, then links and prunes neighbours.
Irregular memory access, locks, and a serial dependency between insertions.

PIPNN (WP3) · NO SEARCH DURING BUILD

Leaves are independent: dense GEMM-like blocks, embarrassingly parallel.
Search is used only at query time; the graph format is our new Lance v3 subindex.

Build cost moves from pointer chasing to matrix math, which is exactly what SVE register tiles accelerate.

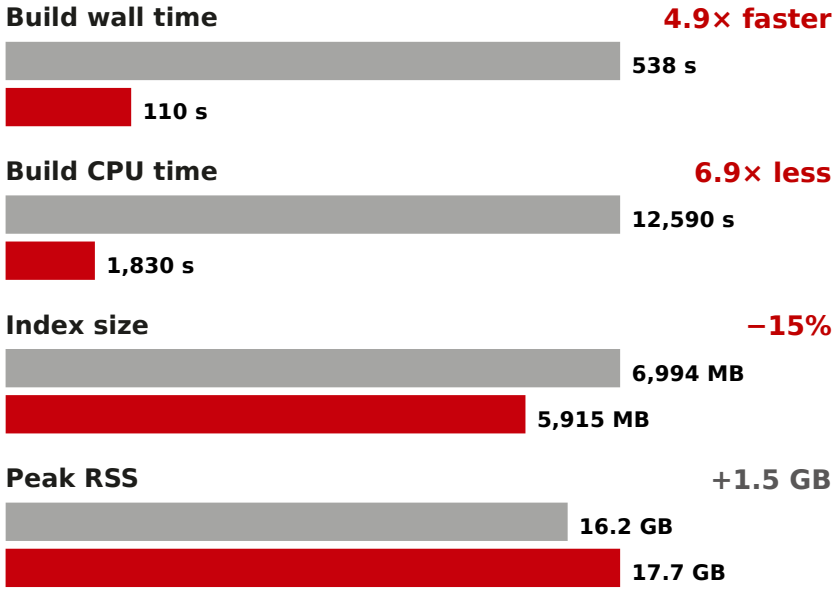
PiPNN turns graph building into kernel work. That is why WP3 depends on WP1 on Kunpeng.

– Huawei Confidential –

PiPNN builds partition graphs 4.9× faster

Proposal 3 · WP3 · [x86] IVF_PIPNN_FLAT vs IVF_HNSW_FLAT, BIGANN10M (10M × 128 f32), K = 5, exact assignment, separate processes, 1 build each

BUILD COST (HNSW GREY · PIPNN RED)



RECALL@10, SAME SETTINGS

nprobes · ef	HNSW	PiPNN	Δ
1 · 32	0.744	0.767	+0.023
1 · 128	0.839	0.848	+0.009
1 · 512	0.860	0.859	−0.001
2 · 32	0.845	0.874	+0.029
2 · 64	0.909	0.935	+0.026
2 · 128	0.949	0.964	+0.014
2 · 512	0.971	0.976	+0.005

Higher at 9 of 10 settings, −0.001 at the 10th (full grid in A2). p50 within a few ms.

STANDALONE CHECK

C++ builders, 10M uint8, 4 threads, no Lance

7.0×

FastPiPNN 222 s vs HNSW 1,557 s

· recall@10 at beam 64: 0.955 vs 0.958
· peak memory 18.8 vs 6.1 GB
· QPS at that point 3,787 vs 5,025

A drop-in replacement for IVF_HNSW_FLAT: same partitions and storage, 4.9× faster and 6.9× cheaper to build, 15% smaller, equal or better recall. The cost is memory: +1.5 GB inside Lance (sequential partitions), up to 3× standalone. WP3 bounds concurrent partitions to a memory budget.

Faster, smaller, as accurate; memory is the trade-off. Tail latency is not claimed (see A2).

– Huawei Confidential –

Kunpeng baseline first; every step has a gate

[Proposal] sizes are rough (S ≈ 1–2 weeks, M ≈ 3–5, L ≈ 6–10 engineer-weeks) · owners to be named · x86 work already done is shown first

PHASES

Done on x86	Phase 1 · Kunpeng bring-up (S)	Phase 2 · P0 SVE tiles (M)	Phase 3 · Integrate on ARM (M)	Phase 4 · Harden + upstream (L)
Harness, suites A/B/C pilots, tree assignment, IVF_PIPNN_FLAT, 6 SVE kernels	Native toolchain, full Lance tests on ARM, SVE vs NEON Criterion, ablation steps 1–3	Tiled L2 + argmin, leaf Gram + top-k; shared by Rust and the PiPNN builder	WP2 + WP3 on Kunpeng with new kernels; frozen suites A/C, ablation steps 4–5	3 repeats, k = 100, full nprobes, per-process RSS, remap / reload tests, Lance PRs
	Gate: baseline table filled	**Gate:** beats old path, scalar parity	**Gate:** speedup at ±0.005 recall	**Gate:** both architectures pass

EXIT / ROLLBACK

· Each kernel sits behind runtime dispatch: if it fails its gate, the old path stays
· Tree and PiPNN are opt-in build options; exact / auto / HNSW remain the defaults until gated

WHAT THE DECISION UNBLOCKS NOW

· Phase 1 needs only Kunpeng server access and one engineer
· Phases 2–3 can run in parallel: kernel owner + integration owner

Small first step, measurable gates, nothing forced on by default. Phase 1 fills every TBD in this deck.

– Huawei Confidential –

Approve WP1 on Kunpeng, then WP2 and WP3

Decision requested from technical leadership · risks are the measured trade-offs, each with a mitigation

Recommendation: fund a gated 3-package project, starting with Kunpeng and the P0 SVE tiles.

WHAT WE KNOW

1. **2.5× IVF_RQ BUILD**
   tree assignment, equal recall (x86)

2. **4.9× PARTITION GRAPHS**
   PiPNN vs HNSW, recall on par or better (x86)

3. **KUNPENG GAP**
   tiles are x86-only; ARM has pairwise kernels

RISKS → MITIGATION

Risk	Mitigation
SVE tile too slow	gate + fallback
PiPNN memory	bound partitions
Approx. assignment	tree:16 / exact
New index format	v3 subindex
Single x86 runs	3 repeats

THE ASK

Kunpeng server time (SVE/SVE2), fixed affinity and NUMA

Owners: WP1 kernels, WP2/WP3 integration

Recall tolerance (proposed ±0.005 recall@10)

Agreement to upstream to Lance

Next 2 weeks after approval: run phase 1 on Kunpeng and return with the filled baseline table (slide 9): SVE vs NEON, and stock vs tree vs PiPNN builds on the same host.

Measured on x86; Kunpeng speed is the open question. One bounded step answers it.

– Huawei Confidential –

Appendix A1 · All assignment methods, both datasets

[x86] AMD EPYC 9124 ×2 · IVF_RQ 5-bit · K = 16,384 · 1 run each · match = same centroid as exact on a 100k sample

BIGANN10M (fixed centroids)	build s	vs exact	assign CPU-s	match	R@10 np16	np64	np256
exact	37.67	1.00×	760.9	1.0000	0.7724	0.9066	0.9430
tree:1	14.96	2.52×	49.5	0.5778	0.7128	0.8808	0.9371
tree:4	**15.11**	**2.49×**	**59.4**	**0.9173**	**0.7680**	**0.9062**	**0.9442**
tree:16	18.97	1.99×	137.3	0.9968	0.7726	0.9083	0.9443
pipnn 15:64:256	18.43	2.04×	174.2	0.9624	0.7651	0.9013	0.9417
pipnn 32:64:256	19.98	1.89×	221.8	0.9931	0.7702	0.9061	0.9428
hnsw 12:15:32	21.77	1.73×	271.4	0.9638	0.7701	0.9060	0.9435
pipnn 64:64:256	23.35	1.61×	296.8	0.9990	0.7710	0.9062	0.9425
hnsw (Lance default)	27.07	1.39×	376.0	0.9520	0.7652	0.9026	0.9423
hnsw 12:15:64	27.38	1.38×	416.2	0.9953	0.7723	0.9068	0.9436
pipnn 128:64:256	27.91	1.35×	493.4	0.9999	0.7726	0.9078	0.9437

GIST1M (centroids retrained)	build s	vs exact	assign CPU-s	match	train s	R@10 np64	np256
exact	31.38	1.00×	384.7	1.0000	9.33	0.7259	0.8984
tree:4	22.97	1.37×	18.9	0.9635	11.79	0.7062	0.8979
auto (Lance default = HNSW)	24.25	1.29×	125.6	—	9.72	0.7192	0.9057

tree:4 is the best speed-quality point; tree:16 is the near-exact option. Sources in notes.

– Huawei Confidential –

Appendix A2 · Recall grid and tail latency

[x86] BIGANN10M, K = 5, recall@10 over 1,970 tuning queries · every completed setting shown

nprobes	ef	HNSW R@10	PiPNN R@10	HNSW p50 ms	PiPNN p50 ms
1	32	0.7444	**0.7669**	4.4	3.3
1	64	0.8023	**0.8214**	4.2	4.1
1	128	0.8391	**0.8478**	7.0	6.3
1	256	0.8545	**0.8565**	5.3	7.5
1	512	0.8601	0.8593	5.9	7.0
2	32	0.8452	**0.8742**	6.1	5.5
2	64	0.9089	**0.9346**	10.0	10.5
2	128	0.9492	**0.9636**	12.2	11.9
2	256	0.9655	**0.9732**	12.9	9.9
2	512	0.9713	**0.9762**	10.2	13.8

WHY P90–P99 ARE NOT REPORTED

· Both indexes showed ~2 s queries: 18% at nprobes 1, 34–37% at 2
· Exactly the same query IDs were slow at every ef → it depends on the partition probed, not on graph search
· Cause: the harness opened the dataset with Lance's default 6 GiB index cache; the indexes are 7.0 GB (HNSW) and 5.9 GB (PiPNN), so a partition is reloaded on every query that probes it
· Affects both indexes alike; build time, memory, size and recall are untouched
· Fix for phase 4: set index cache size explicitly, pre-warm, flag runs with > 1% queries over 100 ms

Recall comparison is complete for nprobes 1–2; tails need a cache fix, not a new index.

– Huawei Confidential –

Appendix A3 · Blocked L2 + constant scale

[x86] GIST1M, 1M × 960, K = 1,000 · 4-core Intel Xeon 2.8 GHz, AVX-512 · 2 repeats, means · fixed nprobes, no reranking

Code width	Config	Build s	IVF train s	Shuffle s	Speedup	R@10 np20	np50	np100
RQ1	baseline	63.6	19.0	44.0	1.00×	0.610	0.656	0.668
RQ1	blocked	30.8	13.3	17.0	**2.07×**	0.614	0.660	0.673
RQ5	baseline	92.2	19.3	70.0	1.00×	0.798	0.912	0.948
RQ5	blocked	56.8	13.0	41.1	1.62×	0.800	0.912	0.949
RQ5	blocked + const	35.0	12.7	19.4	**2.63×**	0.792	0.910	0.946
RQ9	baseline	221.0	18.8	197.3	1.00×	0.804	0.920	0.957
RQ9	blocked	190.3	13.1	172.6	1.16×	0.799	0.917	0.956
RQ9	blocked + const	38.5	12.4	21.9	**5.74×**	0.800	0.919	0.956

How this relates to the project: blocked L2 is the x86 evidence for WP1 tiles (ARM falls back today). Constant scale is algorithmic and portable; it costs at most 0.006 R@10 (RQ5, nprobes 20) and must be checked on more datasets before becoming a default.

This patch is the optimized x86 baseline for every comparison.

– Huawei Confidential –

Appendix A4 · PiPNN memory: model and measurements

Plan §6 first-order model (decimal GB) vs measured peak RSS · PiPNN is the memory trade-off of WP3

FIRST-ORDER MODEL [PROPOSAL]

Term	Formula	N = 2M
input f32 vectors	4·N·D	1.02 GB
HashPrune slots	8·N·S (S = 160)	2.56 GB
final adjacency	4·N·R (R = 64)	0.51 GB
sketches	4·N·16	0.13 GB
subtotal		≈ 4.22 GB
+ leaf scratch	≈ 4·W·L^2	per worker

At 10M rows across concurrently built partitions the same terms reach ≈ 21 GB. Capacity estimate, not RSS.

MEASURED PEAK RSS [X86]

Run	HNSW	PiPNN
Lance, 10M, K = 5, sequential partitions	16.2 GB	17.7 GB
Lance, 1M prefix, K = 5	3.2 GB	5.1 GB
Standalone C++, 10M uint8 (FastPiPNN)	6.1 GB	18.8 GB

1M prefix build time: HNSW 42.3 s vs PiPNN 8.6 s (4.9×), consistent with 10M.

Mitigation in WP3: one memory/CPU budget for active partitions, worker scratch and C++ threads; fail with a recorded capacity result rather than silently shrinking parameters. Kunpeng RAM per node must be recorded.

Memory is bounded by how many partitions build at once.

– Huawei Confidential –

Appendix A5 · Runs, sources and references

All run folders are in the repository with manifest.json (revision, patch hash, machine, resolved options)

ID	Source	Used on
RA	lance_index_build/results/ivf_rq_assignment_results.md (pilotB, pilotA, probeA)	2, 10, 11, A1
RC	lance_index_build/results/pilotC_hnsw_vs_pipnn_results.md (pilotC-bigann10m-k5)	2, 13, A2
MEM	lance_index_build/results/pilotC-mem-1m	A4
RG	lance_ivf_rq/README.md, results/results.jsonl	2, 6, A3
SVE	lance_arm_sve/README.md, lance-arm-sve.patch, scripts/validate_emulated.py, benchmark_native.py	5, 7, 8, 9
NAT	native/results/my-run/summary.json	13, A4
PLAN	LANCE_INDEX_BUILD_PLAN.md (7 Nov 2026)	5, 7–9, 14, A4
P1	Rubel et al., PiPNN, KDD 2026, arXiv:2602.21247v3	12
ADS	ADS-PLAN.pptx slide 1 (internal)	3

Not established yet: any Kunpeng timing · native ARM Rust build · repeats ≥ 3 · k = 100 and nprobes ≥ 3 for suite C · per-process RSS for suite B. Previews may use a substitute font for Microsoft YaHei.

Every number traces to a run folder.

– Huawei Confidential –